

Assignment 17

Submitted by Khairul Basar

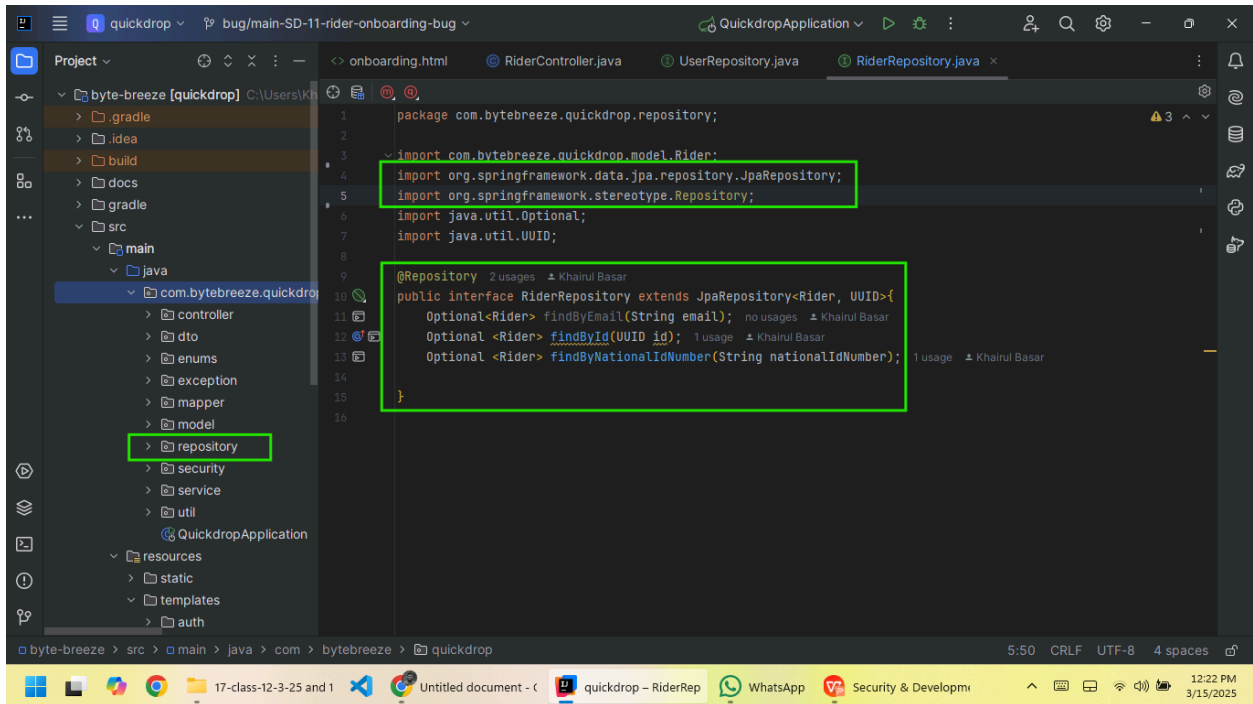
Currently I am working on a java spring boot based project named “[Smart Parcel Delivery System](#)”, where I have addressed and implemented solutions for a few security vulnerabilities based on OWASP Top 10 and VAPT concerns.

Below are the these with screenshots of code:

1. SQL Injection prevention:

To prevent this, I have used ORM(Object Relational Mapping) in java named **JPA Hibernate**. Below is the screenshots:

[Code link](#)

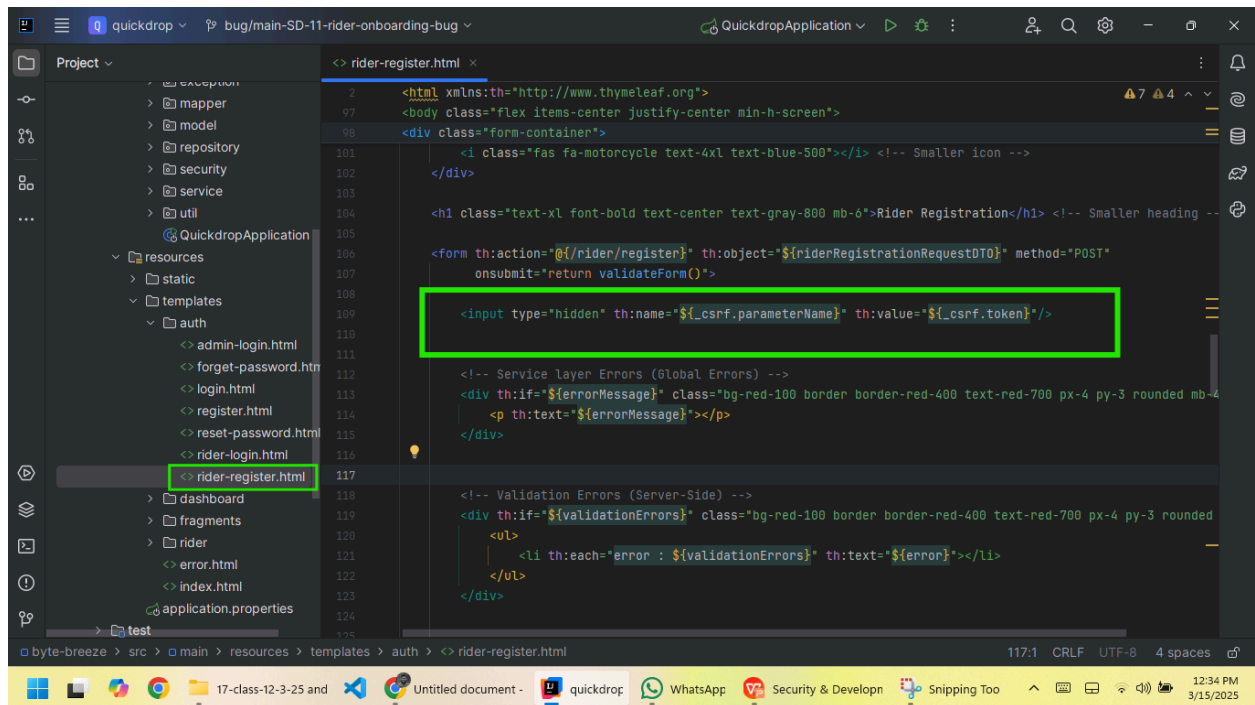


```
1 package com.bytebreeze.quickdrop.repository;
2
3 import com.bytebreeze.quickdrop.model.Rider;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6 import java.util.Optional;
7 import java.util.UUID;
8
9 @Repository
10 public interface RiderRepository extends JpaRepository<Rider, UUID>{
11     Optional<Rider> findByEmail(String email);
12     Optional<Rider> findById(UUID id);
13     Optional<Rider> findByNationalIdNumber(String nationalIdNumber);
14 }
```

2. CSRF prevention:

To prevent this, I have used a **CSRF token** in registration user form design with Thymeleaf template engine. Below is the screenshots:

[Code link](#)



```
<html xmlns:th="http://www.thymeleaf.org">
<body class="flex items-center justify-center min-h-screen">
<div class="form-container">
    <i class="fas fa-motorcycle text-4xl text-blue-500"></i> <!-- Smaller icon -->
</div>

<h1 class="text-xl font-bold text-center text-gray-800 mb-6">Rider Registration</h1> <!-- Smaller heading -->

<form th:action="@{/rider/register}" th:object="${riderRegistrationRequestDTO}" method="POST"
onsubmit="return validateForm()">

    <input type="hidden" th:name="${_csrf.parameterName}" th:value="${_csrf.token}" />

    <!-- Service layer Errors (Global Errors) -->
    <div th:if="${errorMessage}" class="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded mb-4">
        <p th:text="${errorMessage}"></p>
    </div>

    <!-- Validation Errors (Server-Side) -->
    <div th:if="${validationErrors}" class="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded">
        <ul>
            <li th:each="error : ${validationErrors}" th:text="${error}"></li>
        </ul>
    </div>

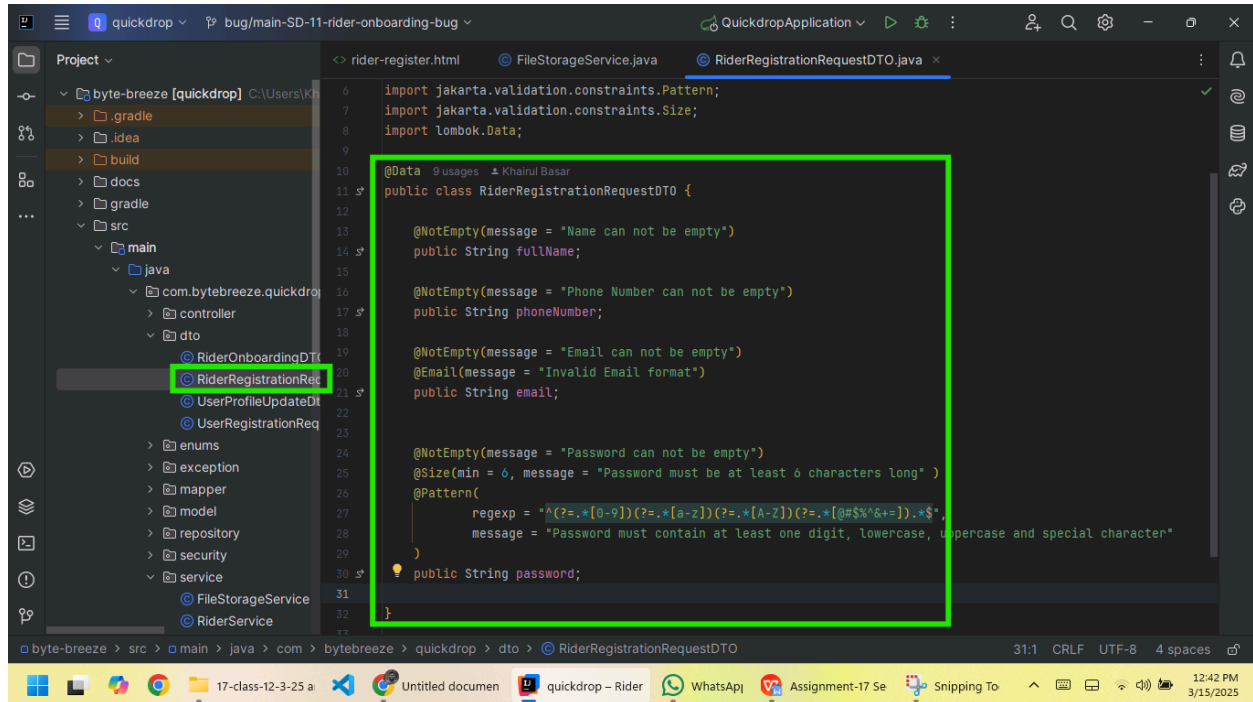
</form>
</body>
</html>
```

3. Sanitize & validate input data:

To validate input data, I have used [DTO](#) (Data Transfer Object) for text data and [File validation](#) for file data in java.

Below are the screenshots:

DTO:



```
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import lombok.Data;

@Data
public class RiderRegistrationRequestDTO {

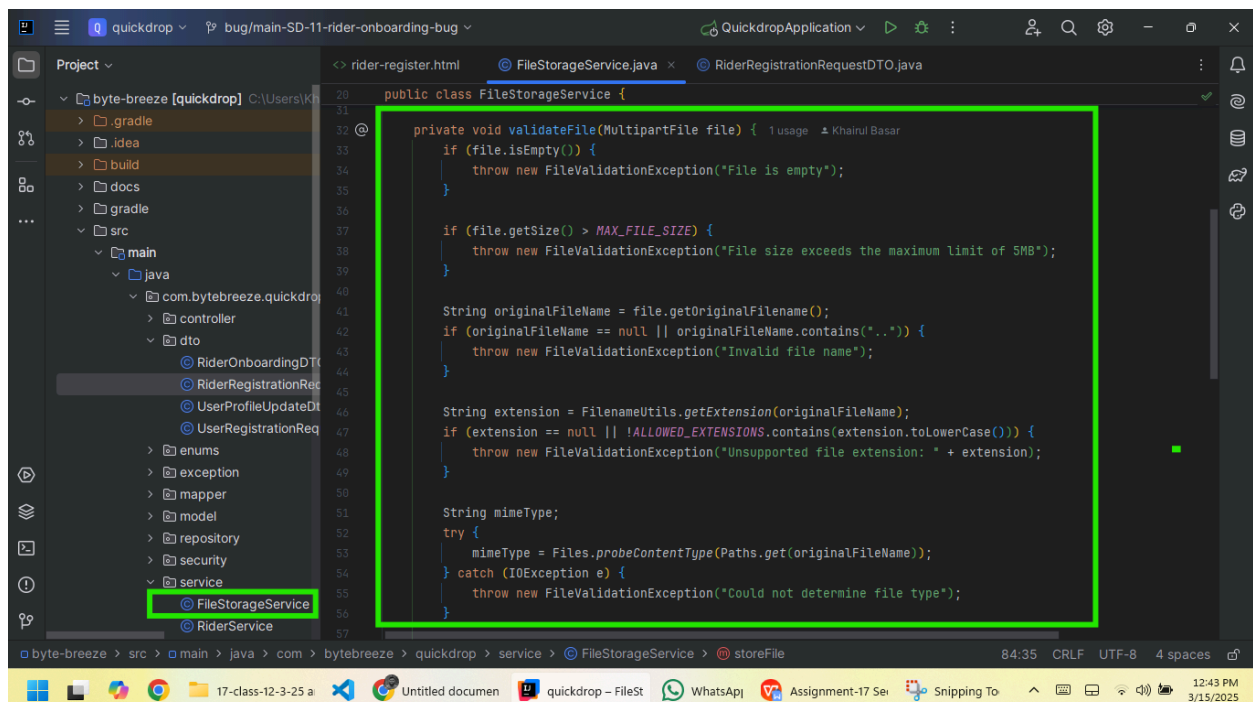
    @NotEmpty(message = "Name can not be empty")
    public String fullName;

    @NotEmpty(message = "Phone Number can not be empty")
    public String phoneNumber;

    @NotEmpty(message = "Email can not be empty")
    @Email(message = "Invalid Email format")
    public String email;

    @NotEmpty(message = "Password can not be empty")
    @Size(min = 6, message = "Password must be at least 6 characters long")
    @Pattern(
        regexp = "^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&+=])$.*$",
        message = "Password must contain at least one digit, lowercase, uppercase and special character"
    )
    public String password;
}
```

File Validation:



```
public class FileStorageService {

    private void validateFile(MultipartFile file) {
        if (file.isEmpty()) {
            throw new FileValidationException("File is empty");
        }

        if (file.getSize() > MAX_FILE_SIZE) {
            throw new FileValidationException("File size exceeds the maximum limit of 5MB");
        }

        String originalFileName = file.getOriginalFilename();
        if (originalFileName == null || originalFileName.contains("..")) {
            throw new FileValidationException("Invalid file name");
        }

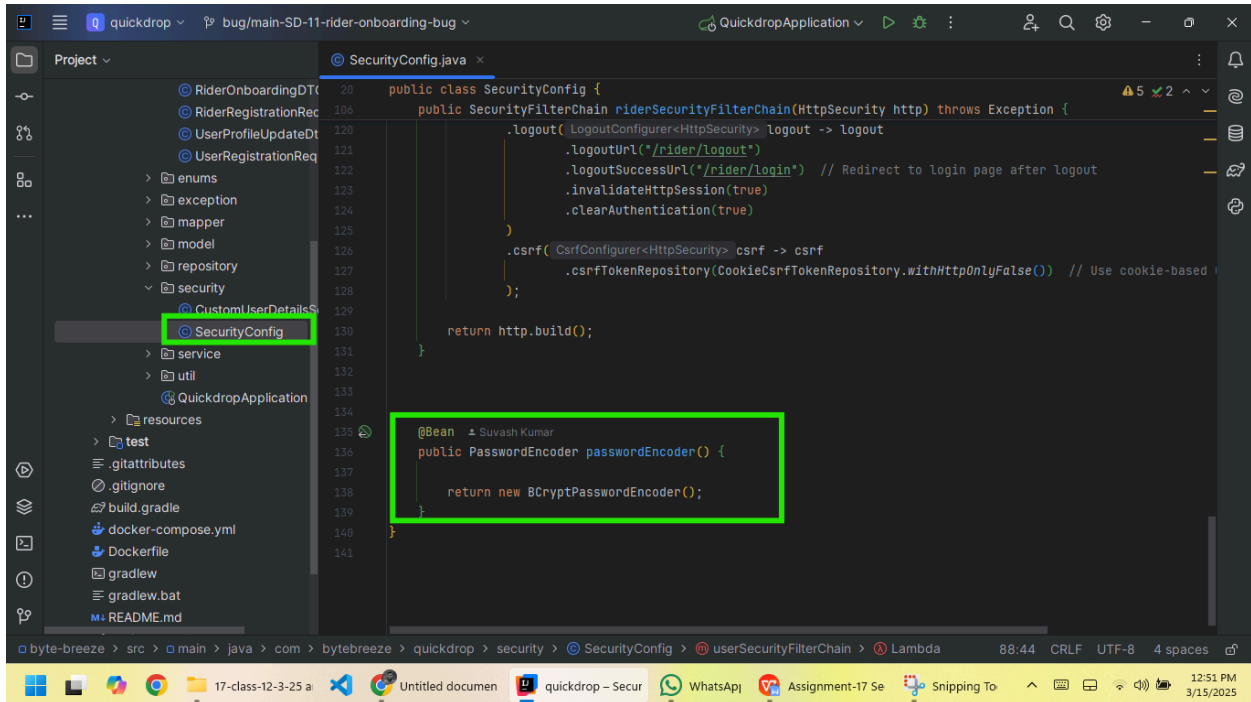
        String extension = FilenameUtils.getExtension(originalFileName);
        if (extension == null || !ALLOWED_EXTENSIONS.contains(extension.toLowerCase())) {
            throw new FileValidationException("Unsupported file extension: " + extension);
        }

        String mimeType;
        try {
            mimeType = Files.probeContentType(Paths.get(originalFileName));
        } catch (IOException e) {
            throw new FileValidationException("Could not determine file type");
        }
    }
}
```

4. Implement secure authentication (bcrypt):

To encrypt the password, I have used a **BcryptPasswordEncoder** and used **Spring boot security**.

[Code](#)

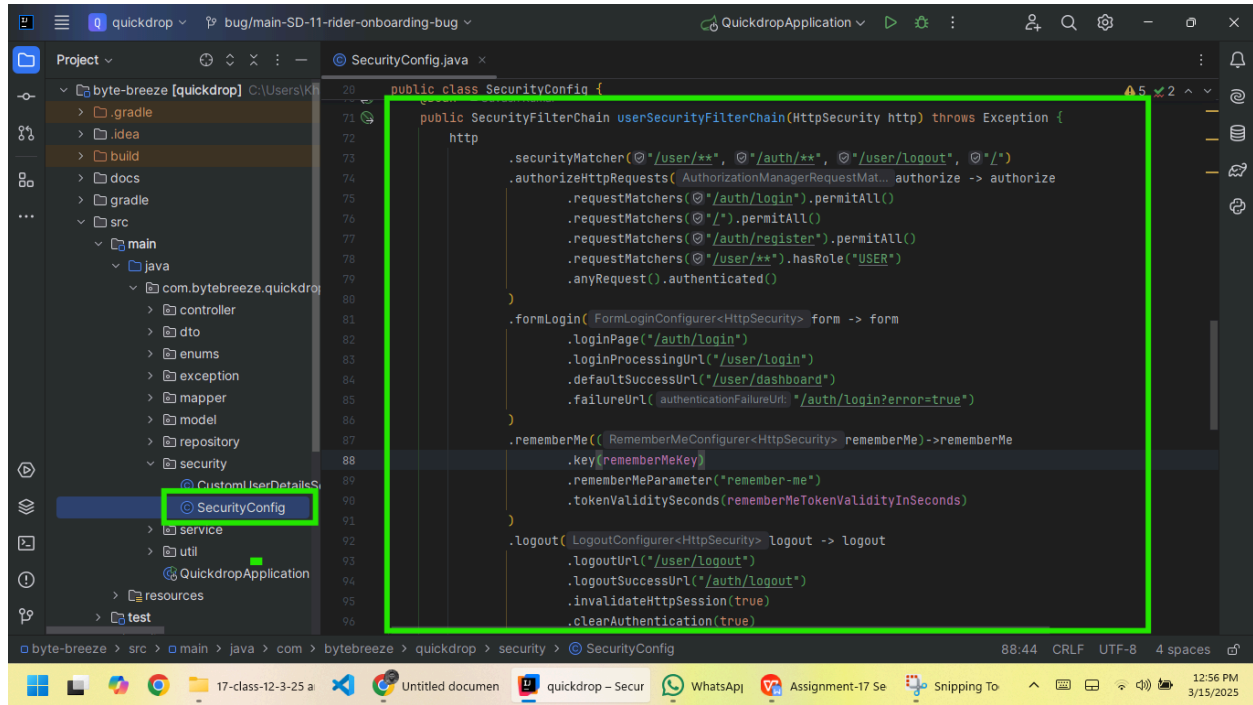


```
public class SecurityConfig {  
    public SecurityFilterChain riderSecurityFilterChain(HttpSecurity http) throws Exception {  
        .logout(LogoutConfigurer<HttpSecurity> logout -> logout  
            .logoutUrl("/rider/logout")  
            .logoutSuccessUrl("/rider/login") // Redirect to login page after logout  
            .invalidateHttpSession(true)  
            .clearAuthentication(true)  
        )  
        .csrf(CsrfConfigurer<HttpSecurity> csrf -> csrf  
            .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse()) // Use cookie-based  
        )  
    }  
    return http.build();  
}  
  
@Bean  
public PasswordEncoder passwordEncoder() {  
    return new BcryptPasswordEncoder();  
}
```

5. Restrict access control using middleware & authorization:

To restrict access control, I have used **FilterChain** (middleware) in Spring Security.

[Code](#)



```
public class SecurityConfig {

    public SecurityFilterChain userSecurityFilterChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher("@{/user/**}", "@{/auth/**}", "@{/user/logout*}", "@/*")
            .authorizeHttpRequests((AuthorizationManagerRequestMatcher) authorize -> authorize
                .requestMatchers("@{/auth/login*}").permitAll()
                .requestMatchers("@/*").permitAll()
                .requestMatchers("@{/auth/register*}").permitAll()
                .requestMatchers("@{/user/**}").hasRole("USER")
                .anyRequest().authenticated()
            )
            .formLogin((FormLoginConfigurer<HttpSecurity> form -> form
                .loginPage("/auth/login")
                .loginProcessingUrl("/user/login")
                .defaultSuccessUrl("/user/dashboard")
                .failureUrl(authenticationFailureUrl: "/auth/login?error=true")
            )
            .rememberMe((RememberMeConfigurer<HttpSecurity> rememberMe -> rememberMe
                .key(rememberMeKey)
                .rememberMeParameter("remember-me")
                .tokenValiditySeconds(rememberMeTokenValidityInSeconds)
            )
            .logout((LogoutConfigurer<HttpSecurity> logout -> logout
                .logoutUrl("/user/logout")
                .logoutSuccessUrl("/auth/logout")
                .invalidateHttpSession(true)
                .clearAuthentication(true)
            )
    }
}
```